

A Self-Certifying Argument of Knowledge

Compiling a proof kernel’s own type checker into a circuit,
and certifying it in zero knowledge

Machine-checked in Lean 4 with Mathlib

Abstract

This note reports a fully machine-checked construction, carried out in Lean 4, in which a formal system produces a zero-knowledge argument about *itself*. The system in question is a constraint-mining framework over a finite labelled object, equipped with a reflected proof kernel and an object language with a typing relation. We (i) give a decision procedure for that typing relation and prove it sound and complete, so that “the types work” becomes a Boolean, exhaustively testable statement; (ii) arithmetize that Boolean statement into an arithmetic circuit and a rank-1 constraint system, and prove that the compiled system is satisfiable exactly when the system’s own type checker accepts a piece of code; (iii) formalize a Σ -protocol argument of knowledge over a linear map, proving completeness, two-transcript special soundness with an explicit extractor, and *perfect* honest-verifier zero knowledge at the level of probability distributions; (iv) equip it with a perfectly hiding commitment and prove that the verifier’s entire view — commitment plus interaction — has a distribution independent of the committed value, so the certified code is never revealed; and (v) assemble these into a single theorem stating the four properties for the self-referential statement. All statements are proved with no `sorry` and depend only on Lean’s standard axioms. The constraint system of a worked catalogue is emitted as JSON by the same Lean definitions whose semantics are proved, so the circuit is an artifact rather than a description.

Contents

1	Introduction	2
1.1	The setting	2
1.2	What “the system works” is taken to mean	2
1.3	Contributions	3
2	The object kernel and its decision procedure	3
2.1	The object language	3
2.2	Inference	3
3	Arithmetization: from the checker to a circuit	4
3.1	Circuits and rank-1 constraint systems	4
3.2	Encoding the secret	4
3.3	The circuit of the type checker	4
3.4	The exported artifact	5

4	The argument of knowledge	6
4.1	The protocol	6
4.2	Zero knowledge, at the level of distributions	6
4.3	A perfectly hiding commitment	7
4.4	Fiat–Shamir	7
5	Self-certification	7
5.1	A concrete certificate	8
5.2	Why this is self-referential	9
6	Scope of the formal claims	9
7	Machine-checked artifact	10
8	Discussion	10

1 Introduction

1.1 The setting

The framework this note builds on treats a finite labelled object $X = \{x_1, \dots, x_n\}$ as sitting underneath an infinite tower of constraints: level 0 records labels and equality; level 1 operations $f_i : X^{k_i} \rightarrow X$ and their closure properties; level 2 invariants $I_j : X^{m_j} \rightarrow V_j$; level 3 relations among invariants; level 4 symmetries; and so on upwards. The object stays finite; only its description grows. Because X is finite, a candidate constraint $c : X^r \rightarrow \{0, 1\}$ can be *tested exhaustively*, so constraints are mined rather than assumed: enumerate, test, retain, generate consequences, test again.

A mined constraint is exactly the kind of statement a zero-knowledge proof system can talk about. It is a decidable predicate on tuples of labels, so “I know a tuple satisfying c ” is an NP statement with a short witness. The natural next question — and the subject of this note — is whether the framework can turn that observation on itself: can the system produce a convincing argument that *it works*, without revealing the object it knows?

1.2 What “the system works” is taken to mean

Three things are formalized under that heading, in increasing depth.

- (a) **The types work.** The object language of the framework is a simply-typed lambda calculus with de Bruijn variables, given by an inductive typing relation $\Gamma \vdash t : A$. We supply a type inference function and prove it sound and complete for that relation, and prove that the resulting Boolean checker agrees with derivability in the reflected rule system used by the proof kernel. Three descriptions of the kernel — inductive predicate, Boolean decision procedure, set-theoretic derivability — are proved to coincide.
- (b) **The math works.** The arithmetization is proved correct: the compiled rank-1 constraint system is satisfiable if and only if the underlying Boolean statement is true. Nothing is assumed about the compiler; both directions are theorems.

- (c) **The argument works.** The Σ -protocol is proved complete, knowledge-sound by an explicit two-transcript extractor, and perfectly zero knowledge, the last as an equality of probability distributions rather than a transcript-by-transcript statement.

The self-referential twist is that the Boolean statement fed to (b) and (c) is the type checker of (a) run on the system’s own object language. The prover convinces a verifier that it possesses a piece of code the kernel accepts, and the verifier finishes the interaction knowing nothing whatsoever about which piece of code that was.

1.3 Contributions

- A decision procedure for the object kernel with soundness, completeness, uniqueness of types, and agreement with the reflected rule system (Section 2).
- An explicit, *runnable* circuit and rank-1 constraint system for the statement “some catalogue entry type-checks at type τ ”, with both directions of correctness proved, and its JSON serialization emitted by the same definitions (Section 3).
- A formalization of the linear (Maurer) preimage Σ -protocol: completeness, special soundness with an explicit extractor, perfect honest-verifier zero knowledge as an equality of PMFs, witness indistinguishability, a perfectly hiding commitment, independence of the joint view from the committed value, and the Fiat–Shamir compilation (Section 4).
- The composite self-certification theorem (Section 5), and an explicit account of what is proved unconditionally and what a computational assumption would be needed for (Section 6).

Everything is machine-checked. Section 7 lists the Lean names, so each claim in this note can be located and re-verified.

2 The object kernel and its decision procedure

2.1 The object language

Types and terms are

$$A, B ::= \text{base } n \mid A \rightarrow B, \quad t ::= \text{var } n \mid tt \mid \lambda A. t,$$

with contexts Γ lists of types and variables de Bruijn indices. The typing relation $\Gamma \vdash t : A$ is the usual inductive one: a variable has the type recorded at its index, an abstraction whose body is typed in the extended context has a function type, and an application requires the function’s domain to match the argument’s type.

The framework already reflects this kernel as a set of rules on judgments and proves that the inductive predicate and set-theoretic derivability from the empty library coincide. What was missing — and what a circuit needs — is a *decision procedure*.

2.2 Inference

Define $\text{infer}(\Gamma, t)$ by structural recursion:

$$\text{infer}(\Gamma, \text{var } n) = \Gamma[n], \quad \text{infer}(\Gamma, \lambda A. b) = A \rightarrow \text{infer}(A :: \Gamma, b),$$

$$\text{infer}(\Gamma, f a) = \begin{cases} B & \text{if } \text{infer}(\Gamma, f) = A \rightarrow B \text{ and } \text{infer}(\Gamma, a) = A, \\ \perp & \text{otherwise,} \end{cases}$$

with the obvious propagation of failure, and set $\text{check}(\Gamma, t, A) := [\text{infer}(\Gamma, t) = A]$.

Theorem 2.1 (`infer_sound`, `infer_complete`). *For all Γ, t, A : if $\text{infer}(\Gamma, t) = A$ then $\Gamma \vdash t : A$; and if $\Gamma \vdash t : A$ then $\text{infer}(\Gamma, t) = A$.*

Corollary 2.2 (`check_iff`, `hasType_unique`, `check_iff_derivable`). *$\text{check}(\Gamma, t, A) = \text{true}$ if and only if $\Gamma \vdash t : A$; consequently typing is decidable, types are unique, and the Boolean checker agrees with derivability of the judgment (Γ, t, A) in the reflected rule system.*

This is the sense in which “the types work” is a theorem here rather than a slogan: the same kernel is available as an inductive predicate for reasoning, as a Boolean function for computation, and as a rule system for reflection, and the three are proved equal.

3 Arithmetization: from the checker to a circuit

3.1 Circuits and rank-1 constraint systems

Over a commutative ring F , an arithmetic circuit is built from constants, input variables, addition and multiplication gates, with the evident evaluation. A rank-1 constraint is a triple $\langle A, B, C \rangle$ of linear combinations of witness variables, read as

$$\langle A, w \rangle \cdot \langle B, w \rangle = \langle C, w \rangle,$$

and a rank-1 constraint system (R1CS) is a list of these. Witness slot 0 carries the constant 1, slot $i + 1$ carries circuit input i , and higher slots are internal wires. Compilation allocates one fresh wire and emits one constraint per multiplication gate; additions are absorbed into linear combinations for free. The framework proves compilation sound (`compile_sound`: a satisfying witness carries on the output combination exactly the value the circuit computes) and complete (`compile_complete`: any input assignment extends to a full witness), hence

$$\exists w, w_0 = 1 \wedge w \models \text{rlcs}(c) \iff \exists x, c(x) = 1.$$

3.2 Encoding the secret

A secret tuple of labels is encoded one-hot: cell $z_{i,x}$ is 1 when position i of the tuple carries label x . Two families of constraints police the encoding: *booleanity* $z \cdot z = z$ for every cell, and *row* $\sum_x z_{i,x} = 1$ for every position. The third family is the acceptance circuit of the mined constraint, forced to output 1.

3.3 The circuit of the type checker

For self-certification the secret is a piece of code, so the finite labelled object is a *catalogue* $\text{cat} : \{0, \dots, m - 1\} \rightarrow \text{LTm}$ of candidate terms, and the mined constraint is the system’s own checker:

$$c_{\tau, \text{cat}}(i) = \text{check}([], \text{cat}(i), \tau).$$

Running the checker on every entry gives the accepted list, and the acceptance circuit is the multilinear polynomial

$$C_{\tau, \text{cat}}(z) = \sum_{i : [] \vdash \text{cat}(i) : \tau} z_i,$$

a sum over exactly those cells whose catalogue entry type-checks. Together with booleanity and the row constraint, this is the system `selfR1CS`.

Theorem 3.1 (`eval_selfCirc_oneHot`). *If $\text{cat}(i)$ has type τ then $C_{\tau, \text{cat}}$ evaluated at the one-hot encoding of i equals 1.*

Theorem 3.2 (`selfR1CS_satisfiable_iff`). *Over any nontrivial commutative ring, `selfR1CS` admits a satisfying witness with $w_0 = 1$ if and only if the catalogue contains a term the typing kernel accepts at type τ .*

Both directions matter. Left-to-right (`hasType_of_selfR1CS_sat`) is the statement that a prover cannot satisfy the circuit by fraud: if no catalogue entry type-checks the acceptance circuit is identically zero and the output constraint demands $0 = 1$. Right-to-left (`selfR1CS_sat_of_hasType`) is the honest prover's strategy: encode the well-typed entry one-hot and extend to the internal wires.

The same equivalence holds for the generic construction that goes through the framework's classical acceptance circuit (`selfSystem_satisfiable_iff`); the explicit version above is stated separately because it is computable and can therefore be exported.

3.4 The exported artifact

The worked catalogue has four entries at target type `base 0 → base 0`: the identity at `base 0`, a stray de Bruijn variable, a self-application, and the identity at the wrong base type. Only the first type-checks — verified by `decide`, i.e. by the kernel evaluating the checker. Over $\mathbb{Z}/7$ the emitted system has six constraints: four booleanity constraints, one row constraint, one output constraint.

```
{
  "field": 7,
  "catalogue_size": 4,
  "accepted_cells": [0],
  "num_constraints": 6,
  "r1cs": [
    {"A": [[1, 1]], "B": [[1, 1]], "C": [[1, 1]]},
    {"A": [[2, 1]], "B": [[2, 1]], "C": [[2, 1]]},
    {"A": [[3, 1]], "B": [[3, 1]], "C": [[3, 1]]},
    {"A": [[4, 1]], "B": [[4, 1]], "C": [[4, 1]]},
    {"A": [[1, 1], [2, 1], [3, 1], [4, 1]], "B": [[0, 1]], "C": [[0, 1]]},
    {"A": [[1, 1], [0, 0]], "B": [[0, 1]], "C": [[0, 1]]}
  ]
}
```

Reading it: rows one to four say each cell is a bit; row five says exactly one cell is set; row six says the acceptance combination equals 1, and since only cell 1 (that is, catalogue entry 0) appears with a nonzero coefficient, the only way to satisfy the system is to set that cell. The file is produced by the Lean definitions of Theorem 3.2, not transcribed by hand.

4 The argument of knowledge

4.1 The protocol

Fix a field $\mathbb{K}$, $\mathbb{K}$ -vector spaces M (finite) and N , and a linear map $\Phi : M \rightarrow N$. The public statement is $y \in N$; the secret is a preimage. The protocol is the linear, or Maurer, preimage Σ -protocol — the common core of Schnorr’s protocol, Guillou–Quisquater, and the opening proofs used by commit-and-prove systems:

1. the prover samples $k \in M$ and sends $a = \Phi k$;
2. the verifier sends a challenge $e \in \mathbb{K}$;
3. the prover replies $z = k + e w$;
4. the verifier accepts iff $\Phi z = a + e y$.

Theorem 4.1 (Completeness, `completeness`). *If $\Phi w = y$ then for every k and every e the transcript $(\Phi k, e, k + ew)$ is accepted.*

Theorem 4.2 (Special soundness, `special_soundness`). *Let (a, e, z) and (a, e', z') both be accepted for y , with $e \neq e'$. Then*

$$\text{Ext}((a, e, z), (a, e', z')) = (e - e')^{-1}(z - z')$$

is a preimage of y , i.e. $\Phi \text{Ext} = y$.

The proof is one line of algebra — subtract the two verification equations to get $\Phi(z - z') = (e - e')y$ and divide — but the content is the standard one: a prover that can answer two distinct challenges on one first message can be rewound into a witness. Knowledge is extracted, not assumed (`knowledge_soundness` states the existential form).

4.2 Zero knowledge, at the level of distributions

Define the simulator, which never sees the secret:

$$\text{Sim}_{y,e}(z) = (\Phi z - e y, e, z).$$

Simulated transcripts always verify. The key identity is that the honest transcript *is* a simulated transcript on re-parameterized randomness:

$$(\Phi k, e, k + ew) = \text{Sim}_{y,e}(k + ew), \quad \Phi w = y,$$

and $k \mapsto k + ew$ is a bijection of M . Since pushing a uniform distribution forward along a bijection returns the uniform distribution (`map_uniformOfFintype_equiv`), we obtain equality of distributions, not merely of supports.

Theorem 4.3 (Perfect HVZK, `perfect_hvzk`). *For every challenge e and every w with $\Phi w = y$,*

$$\text{Real}_{w,e} = \text{Sim}_{y,e} \quad \text{as probability distributions on transcripts,}$$

where $\text{Real}_{w,e}$ is the law of the honest transcript for uniformly random k and $\text{Sim}_{y,e}$ the law of the simulator’s output for uniformly random z .

Corollary 4.4 (Witness indistinguishability, `witness_indistinguishable`). *If $\Phi w = \Phi w' = y$ then $\text{Real}_{w,e} = \text{Real}_{w',e}$: the interaction cannot single out which secret the prover used.*

4.3 A perfectly hiding commitment

Let $\varphi : M \rightarrow N$ and $\psi : R \rightarrow N$ be linear with R finite, and set

$$\text{Com}(w, \rho) = \varphi w + \psi \rho.$$

This is the linear skeleton of a Pedersen commitment: the value contributes φw , the blinding randomness contributes $\psi \rho$.

Theorem 4.5 (Perfect hiding, `perfect_hiding`). *If ψ is surjective then for all w, w' the laws of $\text{Com}(w, \rho)$ and $\text{Com}(w', \rho)$ under uniform ρ coincide. The commitment alone carries no information about what was committed.*

The proof again exhibits a bijection: choose ρ_0 with $\psi \rho_0 = \varphi w - \varphi w'$, and translate.

Running the Σ -protocol on the combined map $\Phi(w, \rho) = \varphi w + \psi \rho$ turns a transcript into a proof of knowledge of an *opening* of the commitment. Combining hiding with zero knowledge gives the statement that matters for the title of this note.

Theorem 4.6 (The code is never revealed, `joint_view_indep_of_witness`). *Assume ψ surjective. For every challenge e and all w, w' , the joint law of*

$$(\text{Com}(w, \rho), \text{ the whole transcript proving knowledge of its opening }),$$

over uniform blinding randomness ρ and uniform protocol randomness k , is the same for w as for w' .

So an unbounded verifier, given the commitment and the entire interaction, cannot distinguish one committed value from another: not with advantage, not negligibly — the two distributions are equal. This is proved by composing two bijections of the randomness space, a shear $(\rho, k) \mapsto (\rho, k + e(w, \rho))$ that absorbs the witness into the response, and a translation of the blinding randomness (`shearEquiv`, `shiftEquiv`).

4.4 Fiat–Shamir

Replacing the verifier’s challenge by a hash $H(y, a)$ of the statement and the first message turns the interaction into a certificate (a, z) verified by

$$\Phi z = a + H(y, a) y.$$

Completeness carries over verbatim (`fs_completeness`), and so does extraction: two certificates with the same first message that verify against different hash values yield a witness (`fs_extraction`), which is the formal shadow of rewinding with a reprogrammed random oracle.

5 Self-certification

Now point the machinery at the system itself. Fix a prime p , the catalogue `cat` and target type τ of Section 3, a public commitment matrix A over $\mathbb{Z}/p$, and take the blinding map to be the identity (surjective, hence perfectly hiding). The prover holds an index i with $[] \vdash \text{cat}(i) : \tau$, encodes it as the one-hot vector `codeVec`(i) — the very vector the circuit’s cells hold — and publishes

$$y = A \cdot \text{codeVec}(i) + \rho.$$

Theorem 5.1 (Self zero-knowledge, `self_zk_argument`). *Let $\text{cat}(i)$ have type τ . Then:*

- (i) the compiled constraint system of the type checker is satisfiable, and any satisfying witness proves that some catalogue entry type-checks;*
- (ii) the honest prover’s transcript is accepted, for every choice of its own randomness and every challenge;*
- (iii) any transcripts accepted on two distinct challenges with the same first message yield, through the explicit extractor, an opening of the published commitment;*
- (iv) for every other catalogue entry j , the verifier’s entire view has exactly the same distribution as it would have had if the prover had committed to j .*

Part (i) is the statement that the circuit means what it should; (ii)–(iii) are the argument-of-knowledge properties; (iv) is the secrecy of the code. The worked instance over $\mathbb{Z}/7$ instantiates all four (`demo_satisfiable`, `demo_sound`, `demo_hides_code`).

5.1 A concrete certificate

Theorem 5.1 is instantiated with numbers so that the argument is an object and not only a statement. Over $\mathbb{Z}/10007$, with public commitment key

$$A = \begin{pmatrix} 3 & 1 & 4 & 1 \\ 5 & 9 & 2 & 6 \end{pmatrix},$$

the prover commits to the one-hot encoding of catalogue entry 0 — the identity function, the only entry that type-checks — blinded by fresh randomness, and applies the Fiat–Shamir transform with a fixed linear stand-in for the random oracle. The published data are

```
{
  "field": 10007,
  "commitment_key": [[3, 1, 4, 1], [5, 9, 2, 6]],
  "statement": [274, 833],
  "first_message": [26, 129],
  "challenge": 2634,
  "response_code": [2636, 7, 1, 8],
  "response_blinding": [3318, 9439]
}
```

and `demoCert_verifies` checks by kernel computation that they satisfy $\Phi z = a + ey$; `demoCert_eq_honest` identifies them as the output of the honest prover algorithm; `demoCert_complete` re-derives acceptance from the general completeness theorem, for arbitrary prover randomness; `demo_extraction` states the knowledge property for this instance; and `demo_hides_code` the secrecy property. Nothing in the published data depends on which entry was committed to, and by Theorem 4.6 nothing in its distribution does either. The hash is a placeholder chosen for reproducibility; every theorem quantifies over an arbitrary hash.

5.2 Why this is self-referential

Three ingredients of the statement are supplied by the system being certified, not by an external oracle:

- the *predicate* being arithmetized is the system’s own type checker, proved equivalent to its own kernel and to its own reflected rule system;
- the *encoding* is the framework’s own one-hot encoding of mined constraints, and the constraint system is emitted by the framework’s own arithmetizer, whose correctness the framework proves;
- the *claim* is about the system’s ability to exhibit well-typed code, which is exactly what the mining loop produces at level 1 of the tower.

The tower’s own vocabulary describes what happens: level 0 is the catalogue of labelled code; level 1 is the typing operation on it; level 2 is the invariant “this entry type-checks”; level 3 relates that invariant to the algebraic identity satisfied by the one-hot cells; and the commitment plus Σ -protocol form a level at which the object of study is the *view* of an observer of the lower levels — and the theorem at that level says the view is constant in the data below it.

6 Scope of the formal claims

Cryptographic claims are easy to overstate, so it is worth being precise about what the machine has checked and what it has not.

Proved unconditionally. Soundness and completeness of the type checker; both directions of the arithmetization; completeness, two-transcript special soundness with an explicit extractor, and perfect HVZK of the Σ -protocol; perfect hiding of the commitment; independence of the joint view from the committed value; completeness and extraction for the Fiat–Shamir certificate. Each of these is a theorem with a proof term, depending only on `proptext`, `Classical.choice` and `Quot.sound`.

Not claimed: binding. A perfectly hiding commitment cannot be information-theoretically binding: every commitment opens to every value under some randomness. Binding, for Pedersen-style commitments, is computational and rests on a hardness assumption. Accordingly the object formalized here is an *argument* of knowledge, not a proof of knowledge — the standard state of affairs, and the reason the word “argument” appears in the title. Theorem 4.2 is unconditional; what a computational assumption would buy is that the extracted opening is the one the prover began with.

Not claimed: a succinct proof of the whole R1CS. The Σ -protocol certifies knowledge of an opening of the commitment; the rank-1 constraint system certifies that a committed one-hot vector encodes well-typed code. Welding the two into a single commit-and-prove argument for the quadratic constraints — so that the verifier is convinced of R1CS satisfaction *and* learns nothing — requires a protocol for multiplicative relations (inner-product arguments, or a polynomial commitment) and is not formalized here. What is formalized is each half, with the shared encoding making the interface explicit.

Not claimed: a soundness-error bound. Special soundness is the standard route to knowledge soundness, and the usual quantitative statement — a cheating prover succeeds with probability at most $1/|\mathbb{K}|$ per repetition — follows from it by a rewinding argument that is not formalized. The worked instance uses $\mathbb{Z}/7$ for legibility; a deployment would use a large prime field, or repeat the protocol.

Not claimed: hiding the statement. The catalogue and the target type are public; what is hidden is which entry the prover holds. Hiding the statement as well would require an OR-composition over the mined constraints, which the present development does not include.

7 Machine-checked artifact

The development is Lean 4 with Mathlib. Table 1 maps the claims of this note to declarations. Every one of them compiles with no `sorry` and uses only the standard axioms; the JSON of Section 3 is produced by running the exporter over the same definitions.

Claim	Lean name	File
Checker decides the kernel	TypeChecker.check_iff	TypeChecker.lean
Checker = reflected rules	TypeChecker.check_iff_derivable	TypeChecker.lean
Uniqueness of types	TypeChecker.hasType_unique	TypeChecker.lean
Circuit semantics	SelfCircuit.eval_selfCirc	SelfCircuit.lean
Honest prover	SelfCircuit.selfR1CS_sat_of_hasType	SelfCircuit.lean
No fraud	SelfCircuit.hasType_of_selfR1CS_sat	SelfCircuit.lean
Circuit correctness	SelfCircuit.selfR1CS_satisfiable_iff	SelfCircuit.lean
Completeness	ZKArgument.completeness	ZKArgument.lean
Special soundness	ZKArgument.special_soundness	ZKArgument.lean
Perfect HVZK	ZKArgument.perfect_hvzk	ZKArgument.lean
Witness indistinguishability	ZKArgument.witness_indistinguishable	ZKArgument.lean
Perfect hiding	ZKArgument.perfect_hiding	ZKArgument.lean
View independent of code	ZKArgument.joint_view_indep_of_witness	ZKArgument.lean
Fiat–Shamir completeness	ZKArgument.fs_completeness	ZKArgument.lean
Fiat–Shamir extraction	ZKArgument.fs_extraction	ZKArgument.lean
Generic self-statement	SelfZK.selfSystem_satisfiable_iff	SelfZK.lean
The composite theorem	SelfZK.self_zk_argument	SelfZK.lean
Concrete certificate verifies	SelfCertificate.demoCert_verifies	SelfCertificate.lean
Certificate is the honest one	SelfCertificate.demoCert_eq_honest	SelfCertificate.lean
Concrete knowledge property	SelfCertificate.demo_extraction	SelfCertificate.lean
Concrete secrecy property	SelfCertificate.demo_hides_code	SelfCertificate.lean

Table 1: Claims of this note and the declarations that establish them.

8 Discussion

Two features of the construction seem worth isolating.

Finiteness is what makes mining and arithmetization the same act. Because X is finite, a candidate constraint can be tested on every tuple; the list of accepted tuples that

the test produces is precisely the list of monomials of the acceptance polynomial. Mining and arithmetization are not two phases but one: the exhaustive test *is* the circuit, read in a different notation. In the self-referential instance the exhaustive test is the type checker running over the code catalogue, and the circuit is the sum of the cells it accepted.

Zero knowledge is a statement about a bijection. Every secrecy theorem above reduces to exhibiting a bijection of the randomness space: a shear that absorbs the witness into the response, and a translation that absorbs the difference of two committed values into the blinding randomness. Once those are named, the distributional statements follow from the single fact that a bijection preserves the uniform distribution. This is why the zero-knowledge claims can be perfect rather than statistical, and why they need no assumption.

Next steps. Three extensions suggest themselves, in increasing order of difficulty: an OR-composition over the mined constraints, which would hide the statement as well as the witness; a commit-and-prove protocol for the quadratic constraints, closing the gap identified in Section 6; and a formal treatment of the soundness error, which would require modelling a rewinding extractor and its success probability.